PreLAB: External Interrupt

Date: 2026-09-28

Author/Partner: Won Bitna

Demo Video: https://youtu.be/1l5tOc4s6R0

Introduction

In this tutorial, we will learn how to use External Interrupt. We will create functions that capture the falling edge trigger by pushing a button using an external interrupt.

The objectives of this tutorial are how to

- Configure External input (EXTI) interrupt with NVIC
- Create your own functions for configuration of interrupts

Submit the Pre-Lab report

Hardware

- NUCLEO -F411RE

Software

- VS code, CMSIS, EC_HAL

Documentation

- STM32 Reference Manual

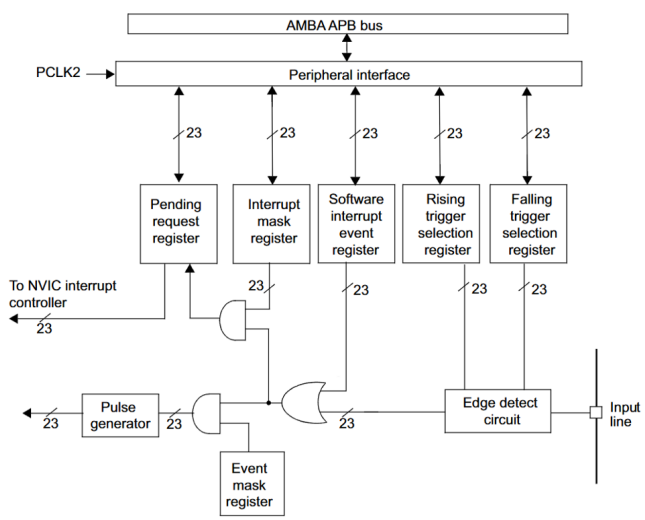
Overview of External Interrupt (EXTI)

A. Register List

List of external interrupt (EXTI) registers

- **EXTI Register List**
  - **Control register**
    SYSCFG_EXTICR
  - **Mask register**
    EXTI_IMR
    EXTI_EMR
  - **Trigger registers**
    EXTI_FTSR
    EXTI_RTSR
  - **Pending register**
    EXTI_PR

Reference Manual : Ch 7, Ch10



```
typedef struct
{
  __IO uint32_t IMR;    // EXTI Interrupt mask register
  __IO uint32_t EMR;    // EXTI Event mask register
  __IO uint32_t RTSR;   // EXTI Rising trigger selection register
  __IO uint32_t FTSR;   // EXTI Falling trigger selection register
  __IO uint32_t SWIER;  // EXTI Software interrupt event register
  __IO uint32_t PR;     // EXTI Pending register
}EXTI_TypeDef
```

B. Register Setting

1. GPIO:Select & Initialize GPIO Port*y*Pin***x* **for EXTIx**

- mode = INPUT

2. EXTI Setting

- Enable SYSCFG peripheral clock. **RCC->APB2ENR**
- Select GPIO Port *y* to EXTI: **SYSCFG->EXTICR**
- Configure the trigger edge. **EXTI->FTSR or RTSR**
- Enable EXTI: **EXTI->IMR**
  - Not-mask Interrupt request

3. NVIC

- `NVIC_SetPriority()`
- `NVIC_EnableIRQ()`

4. EXTI handler

- is_pending ? Then `EXTIx_IRQHandler()`
- clear_pending when finished

Exercise

A. Exercise 1: Register Configuration

Fill in the blanks below

Assumption:

`#define BUTTON_PIN PA_4 //EVAL board JKIT`

EXTI GPIO Pin initialization

- Push-Button: Port A Pin 4 / Input / PU

```
// Use your API library  GPIO
GPIO_init(PA_4, INPUT);
GPIO_pupd(PA_4, PULL_UP);
```

EXTI Configuration

1. **Enable EXTI Clock with SYSCFG controller clock**

- **RCC_APB2ENR:** SYSCFGEN

```
// SYSCFG peripheral clock enable
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
```

6.3.12 RCC APB2 peripheral clock enable register (RCC_APB2ENR)

Address offset: 0x44

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved											SPI5EN	Reser-ved	TIM11 EN	TIM10 EN	TIM9 EN
											rw		rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reser-ved	SYSCFG EN	SPI4EN	SPI1 EN	SDIO EN	Reserved		ADC1 EN	Reserved		USART6 EN	USART1 EN	Reserved			TIM1 EN
	rw	rw	rw	rw			rw			rw	rw				rw

Bit 14 **SYSCFGEN:** System configuration controller clock enable
Set and cleared by software.
0: System configuration controller clock disabled
1: System configuration controller clock enabled

2.**Connect EXTI to the GPIO Port_y Pin_x**

- **SYSCFG_EXTICR2:** EXTIx
- Connect PA_4(push-button) to EXTI4 line (given code)

```
// Button: PA_4 -> EXTICR2(EXTI4)
// Pin 4~7: EXTICR2 // EXTICR[1]

SYSCFG->EXTICR[1] &= ~(0xFUL << 0);
SYSCFG->EXTICR[1] |=  0UL<<0;

// These are the same as
SYSCFG->EXTICR[1] &= ~SYSCFG_EXTICR2_EXTI4;
SYSCFG->EXTICR[1] |= SYSCFG_EXTICR2_EXTI4_PA;
```

7.2.6 SYSCFG external interrupt configuration register 4 (SYSCFG_EXTICR4)

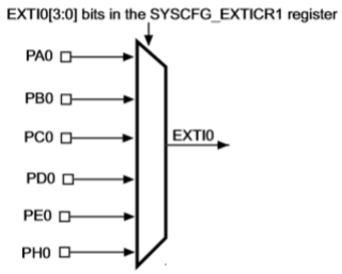
Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 12 to 15)
These bits are written by software to select the source input interrupt.
0000: PA[x] pin
0001: PB[x] pin
0010: PC[x] pin
0011: PD[x] pin
0100: PE[x] pin
0101: Reserved
0110: Reserved
0111: PH[x] pin



- **Generalize of Connecting Port y, Pin x to EXTIx**

```
// Connect EXTI to the GPIO Port_y Pin_x
    // Pin 0~3: EXTICR1 // EXTICR[0]
    // Pin 4~7: EXTICR2 // EXTICR[1]
    // Pin 8~11: EXTICR3 // EXTICR[2]
    // Pin 12~15: EXTICR4 // EXTICR[3]

    uint32_t EXTICR_port=0;
    if      (Port == GPIOA) EXTICR_port = 0;
    else if (Port == GPIOB) EXTICR_port = 1;
    else if (Port == GPIOC) EXTICR_port = 2;
    else if (Port == GPIOD) EXTICR_port = 3;
    else                    EXTICR_port = 4;

    SYSCFG->EXTICR[pin/4] &= ~(0xFUL<<(pin%4)*4);            // clear 4 bits
    SYSCFG->EXTICR[pin/4] |= (EXTICR_port<<(pin%4)*4);          // Set 4
bits
```

> HINT 1 : mod(pin,4) * 4, (int)(pin/4)
>
> HINT 2 : Find the pattern in the binary representations

Bit Starting Position	**Pin y**		
	EXTICR[0]	**EXTICR[1]**	**EXTICR[2]**
Bit 0	0 (b00**00**)	4 (b01**00**)	8 (b10**00**)
Bit 4	1 (b00**01**)	5 (b01**01**)	9 (b10**01**)
Bit 8	2 (b00**10**)	6 (b01**10**)	10 (b10**10**)
Bit 12	3 (b00**11**)	7 (b01**11**)	11 (b10**11**)

3. **Select Trigger Edge (RISE, FALL, BOTH)**

- **EXTI_FTSR:** TRx or **EXTI_RTSR:** TRx

```
    // Select  Trigger edge (RISE, FALL, BOTH)
    if (trig_type == FALL)      EXTI->FTSR |= 1UL << pin;  // Falling
trigger enable
    else if (trig_type == RISE) EXTI->RTSR |= 1UL << pin;   // Rising
trigger enable
    else if (trig_type == BOTH) {           // Both falling/rising trigger
enable
        EXTI->RTSR |= 1UL << pin;
        EXTI->FTSR |= 1UL << pin;
```

10.3.3 Rising trigger selection register (EXTI_RTSR)

Address offset: 0x08
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									TR22	TR21	Reserved		TR18	TR17	TR16
									rw	rw			rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:0 **TRx:** Rising trigger event configuration bit of line x
0: Rising trigger disabled (for Event and Interrupt) for input line
1: Rising trigger enabled (for Event and Interrupt) for input line

Note: *The external wake-up lines are edge triggered, no glitch must be generated on these lines. If a rising edge occurs on the external interrupt line while writing to the EXTI_RTSR register, the pending bit is be set.*

Rising and falling edge triggers can be set for the same interrupt line. In this configuration, both generate a trigger condition.

4. **Enable EXTI: Not-masking Interrupt request**

- **EXTI_IMR:** MRx
    - Enabling == not-masking

```
    // Enable EXTI, with Not-Masked Interrupt Request
    // Not-mask (==Enable) EXTIx
    EXTI->IMR |= 1UL << pin;
```

10.3.1 Interrupt mask register (EXTI_IMR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									MR22	MR21	Reserved		MR18	MR17	MR16
									rw	rw			rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:0 **MRx:** Interrupt mask on line x
0: Interrupt request from line x is masked
1: Interrupt request from line x is not masked

EXTI_NVIC

- Search for "**Vector table for STM32F411xC/E** " in Reference Manual
- Fill-in the blanks in the table for EXTI 0 to EXT 15

EXTIx	ISR (handler)	IRQn (keyword, stm32f411xe.h)	IRQn
EXTI0	EXTI0_IRQHandler()	EXTI0_IRQn	6
EXTI1	EXTI1_IRQHandler()	EXTI1_IRQn	7
EXTI2	EXTI2_IRQHandler()	EXTI2_IRQn	8
EXTI3	EXTI3_IRQHandler()	EXTI3_IRQn	9
EXTI4	EXTI4_IRQHandler()	EXTI4_IRQn	10
EXTI5_9	EXTI9_5_IRQHandler()	EXTI9_5_IRQn	23
EXTI10_15	EXTI15_10_IRQHandler()	EXTI15_10_IRQn	40

- Fill-in the blanks

```
// NVIC(IRQ) Setting
uint32_t EXTI_IRQn = 0;
if (pin < 5)                EXTI_IRQn = EXTI0_IRQn + pin;
else if (pin < 10)  EXTI_IRQn = EXTI9_5_IRQn;
else                                EXTI_IRQn = EXTI15_10_IRQn;

NVIC_SetPriority(EXTI_IRQn, priority);
NVIC_EnableIRQ(EXTI_IRQn);
```

EXTI_handler

1. **Check for Pending Status**
    - **EXTI_PR:** PRx
    - Read the bit at PRx and returns either 1 or 0

```
uint32_t is_pending_EXTI(PinName_t pinName) {
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    uint32_t EXTI_PRx = (EXTI->PR>>pin) & 1UL;      // read PRx, EXTI pending
bit.  Use  (REG>>k & 1) to read.
    return EXTI_PRx;                                // Return only 0 or 1
}
```

2. **Clear Pending **
    - **EXTI_PR:** PRx
    - By writing '1', it clears the pending bit.

```
void clear_pending_EXTI(PinName_t pinName){
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName, &Port, &pin);

    // clear by writing 1 to the pending bit
    EXTI->PR  |= (1 << pin);
}
```

B. Exercise 2 : Programming

This is an example code for toggling LED on/off with the button input trigger (EXTI)

- Fill in the empty spaces in the downloaded library `ecEXTI2.c`
- Run the program and check your result.
- Your tutorial report must be submitted to LMS

Procedure

- Download the header library files and save under `include\`.
    - `ecEXTI2_student.h, ecEXTI2_student.c` : Click here to download
- Rename the files as `ecEXTI2.h, ecEXTI2.c`

> Write your name and modified date in the comment box

- Create a new project under the directory `EC\tutorial\`
    - Environment : `env:TU_EXTI`
    - Source file: `TU_EXTI_22400462.c`
- Modify the `platformio.ini` , to add new environment.

Complete the function `led_toggle(void)`

> You MUST write your name on the source file inside the comment section

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Won Bitna
Created          : 05-03-2021
Modified         : 09-29-2026
Language/ver     : C++ in VS Code

Description      : Tutorial - PreLAB: External Interrupt
/----------------------------------------------------------------*/


#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"
#include "ecEXTI2.h"

#define LED0_PIN   PB_12        //EVAL board JKIT LED0
#define LED7_PIN   PC_3         //EVAL board JKIT LED7
#define SW2_PIN    PA_4         //EVAL board JKIT SW2


void led_toggle(PinName_t pinName){
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    // YOUR CODE GOES HERE - Use XOR
    Port->ODR ^= (1UL << pin);
}

// Initialiization
void setup(void)
{
        RCC_PLL_init();                 // System Clock = 84MHz
        SysTick_init();                 // SysTick Timer Initialization
        // Initialize GPIOB_12 for Output
        GPIO_init(LED0_PIN, OUTPUT);    // LED0 for EVAL board
        GPIO_init(LED7_PIN, OUTPUT);    // LED7 for EVAL board

        // Initialize GPIOA_4 for Input Button
        GPIO_init(SW2_PIN, INPUT);  // INPUT for EVAL board
        GPIO_pupd(SW2_PIN, PULL_UP);  // PULL-UP for EVAL board

        // Initialize EXTI PA_4
        EXTI_init(SW2_PIN, FALL, 10);
}



// MAIN  ----------------------------------------
int main(void) {
    setup();
    while (1){
        GPIO_write(LED7_PIN, HIGH);
        delay_ms(1000);
        GPIO_write(LED7_PIN, LOW);
        delay_ms(1000);
```

```
    }
}


void EXTI4_IRQHandler(void) {
    if (is_pending_EXTI(SW2_PIN)) {
        led_toggle(LED0_PIN);
        clear_pending_EXTI(SW2_PIN);
    }
}
```